
Architecture Planning Document

A Compact Discriminative Code Review Model Specialised for Next.js

Project type	Research proposal — model design and training plan
Focus domain	Next.js App Router (TypeScript / React Server Components)
Model family	Discriminative encoder (CodeBERT / GraphCodeBERT) + LoRA
Date	March 2026

ABSTRACT

This document presents the complete architectural plan for a compact, locally deployable discriminative model for automated code review, specialised for the Next.js framework. The system fine-tunes a CodeBERT or GraphCodeBERT encoder using LoRA on a curated dataset of real GitHub pull request diffs, producing four simultaneous outputs: multi-label issue category classification, a quality score, a severity rating, and line-level issue localisation. The quantised model is approximately 50 MB and runs at sub-150ms inference on CPU only — requiring no internet connection, no cloud service, and no GPU. The document covers input design, encoder architecture, multi-task training, dataset construction, evaluation strategy, and deployment tooling across nine planned project phases.

Keywords: automated code review · discriminative classification · CodeBERT · LoRA fine-tuning · Next.js · multi-task learning · local deployment

1 System Overview

End-to-end pipeline from data collection to deployed developer tool

The system is structured as a single pre-trained transformer encoder fine-tuned on domain-specific data, with all inference running on the developer's local machine. No code leaves the local environment. Figure 1 shows the complete pipeline across six stages.

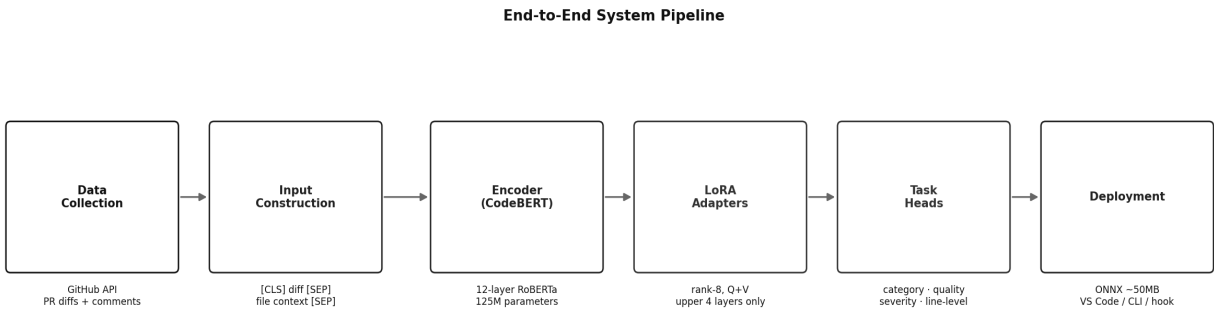


Figure 1. End-to-end pipeline: GitHub PR data collection input construction CodeBERT encoder LoRA fine-tuning task heads ONNX deployment.

The six pipeline stages are: (1) collecting real pull request diffs and inline reviewer comments from high-activity Next.js repositories via the GitHub REST API; (2) constructing the tokenised input sequence by concatenating the file path, diff, and surrounding file context; (3) encoding the sequence through the pre-trained CodeBERT or GraphCodeBERT transformer; (4) applying LoRA adapters to the upper transformer layers during fine-tuning; (5) routing encoder outputs to four task-specific heads; and (6) exporting to ONNX with 4-bit quantisation for local deployment.

2 Input Representation

Token sequence construction, segment embeddings, 512-token budget

The input encodes three complementary information sources: the file path, the code diff, and surrounding file context. For Next.js specifically, file context is not optional — whether a component is a Server or Client Component is determined by the 'use client' directive at the file top, which does not appear in a diff of a changed function within that file.

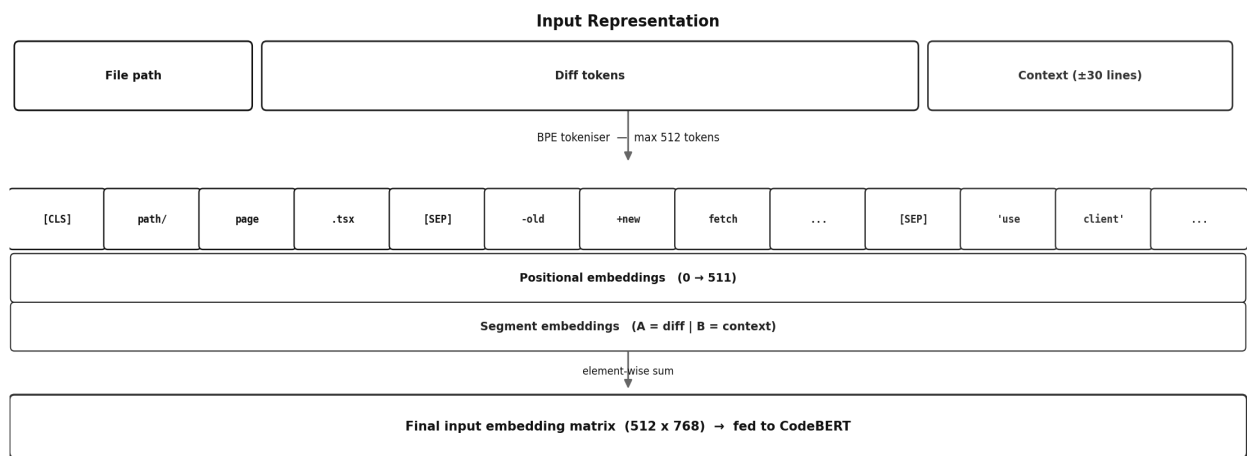


Figure 2. Input token sequence construction. Three sources are concatenated with [CLS] / [SEP] separators, summed with positional and segment embeddings, and passed to the encoder.

2.1 Token sequence structure

The sequence follows the BERT two-segment format: [CLS] [SEP] [SEP]. The [CLS] token produces the aggregate representation consumed by the three document-level heads. Per-token representations feed the line-level head. The 512-token budget is allocated as: file path (fewer than 10 tokens), diff (up to 350 tokens), and context (approximately 30 surrounding lines). Over 92% of TypeScript diffs in the target repositories fit without truncation.

2.2 Why file context is necessary for Next.js

Issue category	Missing from diff alone	Present in file context
nextjs-pattern	Server vs. Client Component boundary	'use client' at file top
security	Whether Server Action validates its caller	'use server' + auth guard pattern
performance	Whether fetch is inside a cached route segment	export const revalidate in layout.ts

Table 1. Three categories that require file context for correct classification. Without context, all three diffs appear ambiguous.

3 Encoder Architecture and LoRA Fine-tuning

12-layer RoBERTa encoder, layer-freezing strategy, rank-8 adapters

The encoder backbone is either CodeBERT or GraphCodeBERT — both RoBERTa-based architectures with 12 transformer layers, 12 self-attention heads, a 768-d hidden dimension, and approximately 125 million parameters. They differ in pre-training: CodeBERT uses masked language modelling on NL-PL pairs; GraphCodeBERT additionally incorporates data flow graphs, giving it stronger structural code understanding for tasks requiring variable dependency reasoning.

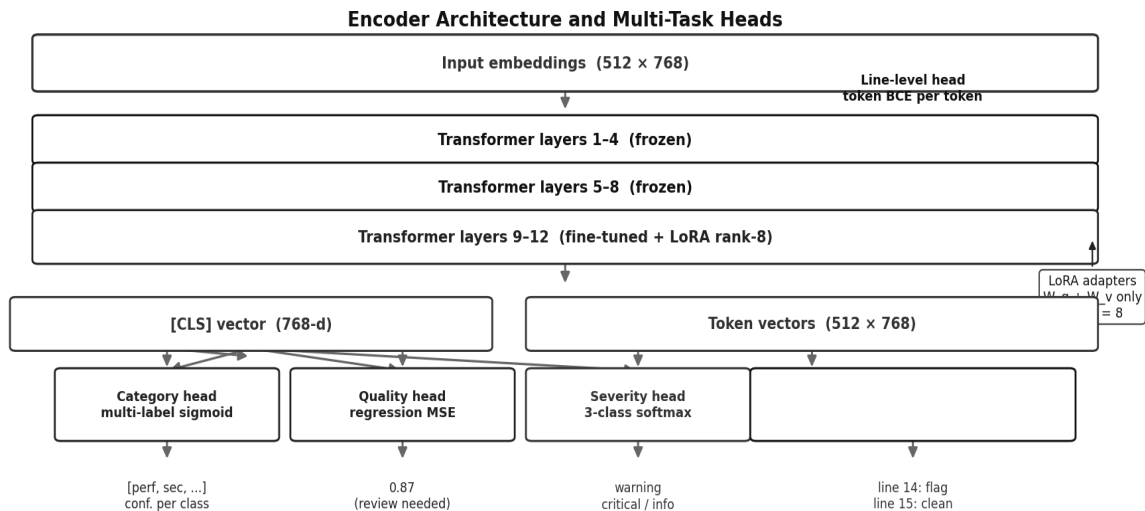


Figure 3. Encoder internals with LoRA and task heads. Layers 1-8 are frozen; layers 9-12 receive rank-8 LoRA adapters on W_q and W_v . The [CLS] token drives three sequence-level heads; all token vectors drive the line-level classification head.

3.1 Layer-freezing strategy

Research by Shi et al. (2023) established that lower transformer layers encode general syntactic code properties while upper layers encode semantic, task-specific features. Layers 1-8 are therefore frozen during fine-tuning and layers 9-12 receive LoRA adapters. This reduces trainable parameters from 125M to approximately 300K – less than 0.25% of total parameters – while preserving the general code understanding acquired during pre-training on hundreds of millions of code-text pairs.

3.2 LoRA adapter configuration

```
rank (r)      : 8
alpha (scaling) : 16
target modules : q_proj, v_proj (attention only)
layers        : 9, 10, 11, 12 (upper 4 of 12)
trainable params : 4 layers x 2 matrices x 2 x (768 x 8) = 196,608
initialisation  : A ~ N(0, sigma), B = 0 (zero initial perturbation)
```

With rank $r=8$ and hidden dimension $d=768$, each LoRA adapter contributes $2 \times 768 \times 8 = 12,288$ parameters per layer. Applied to Q and V weight matrices in the top four layers, the total LoRA overhead is 196,608 parameters — less than 0.16% of the base model size. The zero initialisation of matrix B ensures the model begins fine-tuning exactly at the pre-trained checkpoint with no initial perturbation.

4 Multi-Task Output Heads

Four simultaneous outputs from one shared encoder

Four task-specific heads operate on the shared encoder output. Three heads use the [CLS] aggregate vector; one uses all token vectors. All heads are single linear projections followed by an activation — shallow by design to keep inference fast.

Head	Input	Dim out	Activation	Loss
Category	[CLS] 768-d	8 logits	Sigmoid	Binary cross-entropy per class
Quality	[CLS] 768-d	Scalar	Sigmoid	Mean squared error (MSE)
Severity	[CLS] 768-d	3 logits	Softmax	Cross-entropy
Line-level	All tokens 512x768	1/token	Sigmoid	Binary cross-entropy per token

Table 2. Four multi-task heads. All share one encoder; no encoder weights are duplicated.

4.1 Combined training objective

$L_{total} = w1 * L_{category} + w2 * L_{quality} + w3 * L_{severity} + w4 * L_{lines}$

Initial weights : $w1 = w2 = w3 = w4 = 1.0$
Grid search over : $w1$ in {0.5, 1.0, 2.0} | $w4$ in {0.5, 1.0, 2.0}
Alternative : uncertainty weighting (Kendall et al., CVPR 2018)

WHY MULTI-TASK LEARNING HELPS HERE

Issue category, severity, and quality are correlated signals. A security issue is almost always critical; a naming issue is almost invariably informational. Training all heads simultaneously forces the shared encoder to build representations that capture these correlations, improving performance on each individual head beyond what single-task fine-tuning achieves — especially for low-frequency categories where training data is scarce.

5 Dataset Construction

GitHub PR collection, filtering, label extraction, splitting

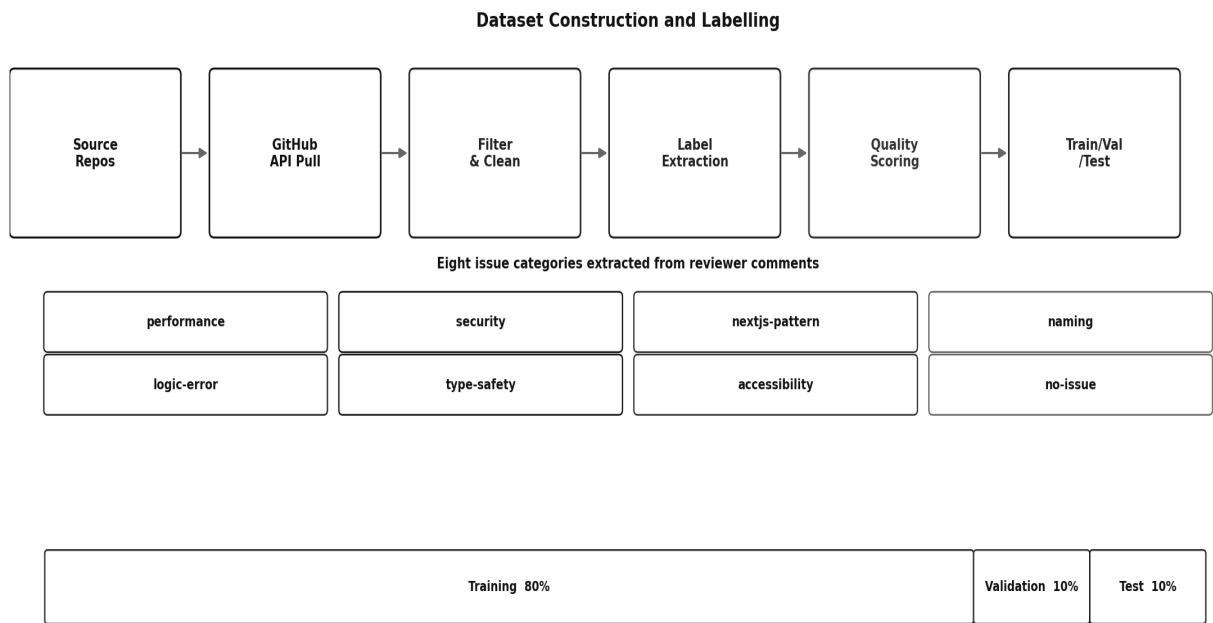


Figure 4. Dataset construction and labelling pipeline. Source repositories are queried via the GitHub REST API; raw PR data is filtered; labels are extracted from inline reviewer comments; the dataset is split by PR to prevent leakage.

5.1 Collection and filtering

Closed pull requests are retrieved using `GET /repos/{owner}/{repo}/pulls?state=closed`. Per PR: the `diff` (`Accept: application/vnd.github.v3.diff`) and `inline comments` (`GET /pulls/{pull_number}/comments`). Inclusion criteria: only `.ts` and `.tsx` diffs; comment body length ≥ 40 characters; comment author type $\neq$ Bot; comment body does not match `/^(LGTM|looks good|nit:)/i`; diff contains at least 3 changed lines. Target: 20,000-30,000 raw samples yielding 15,000-20,000 after filtering.

5.2 Label extraction

Labels are extracted in two passes. Pass 1 applies deterministic keyword patterns (e.g. 'useCallback', 're-render' -> performance; 'exposed', 'sanitize' -> security; 'Server Component', 'App Router' -> nextjs-pattern). Pass 2 sends unmatched samples to GPT-4o with a structured JSON-output prompt. A 10% random sample of all auto-labelled examples is manually verified to establish label reliability before training begins.

5.3 Train / validation / test split

The dataset is split at the PR level (not the comment level) using an 80/10/10 ratio. Splitting by PR prevents data leakage: comments from the same PR share diff context and reviewer style, so placing them across splits would inflate test performance unrealistically. Stratified sampling is used to ensure each split preserves the overall class distribution across all eight issue categories.

6 Training Setup

Hyperparameters, schedule, hardware requirements

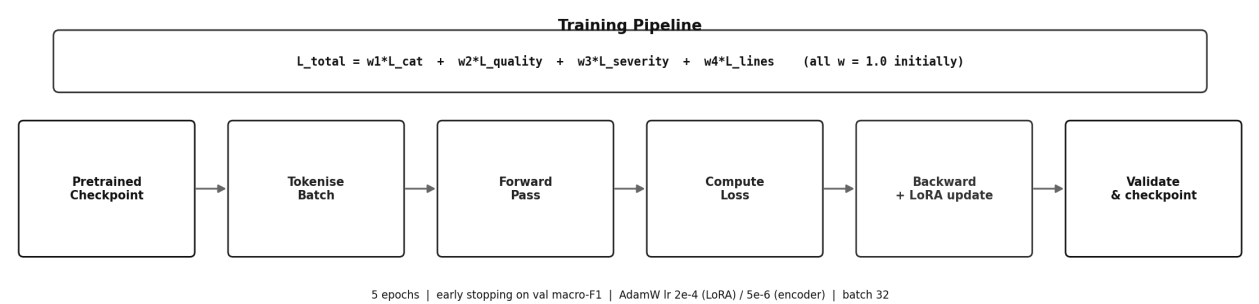


Figure 5. Training pipeline with loss equation and per-step details.

Base model	microsoft/codebert-base or microsoft/graphcodebert-base
Tokeniser	RobertaTokenizer max_length=512 truncation=True
LoRA rank / alpha	r=8 alpha=16 target=q_proj,v_proj layers 9-12
Batch size	32 (gradient accumulation steps=4 if VRAM < 8GB)
Learning rate	2e-4 (LoRA params) 5e-6 (unfrozen encoder layers)
LR schedule	Linear warmup 500 steps then cosine decay
Epochs	5 early stopping on val macro-F1 patience=2
Optimiser	AdamW beta1=0.9, beta2=0.999 weight_decay=0.01
Min hardware	8 GB VRAM GPU or CPU-only (slower, ~4x longer)
Libraries	transformers + peft + bitsandbytes (HuggingFace stack)

Table 3. Training hyperparameter configuration.

7 Local Deployment Architecture

ONNX export, quantisation, VS Code / CLI / pre-commit integration

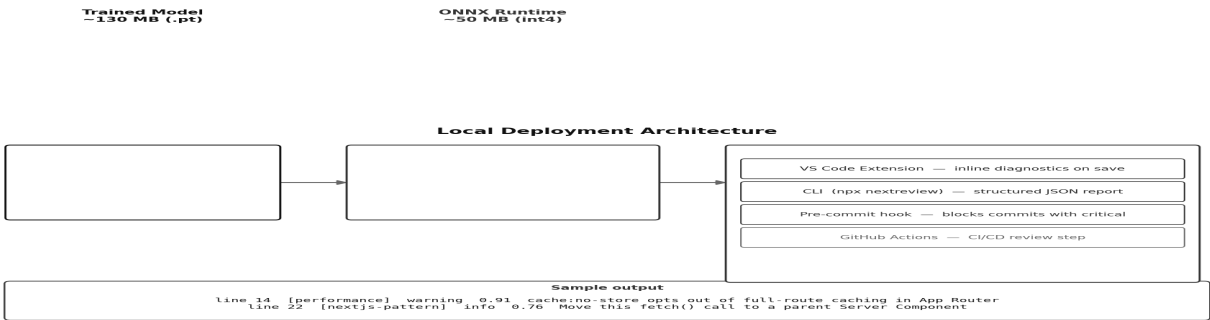


Figure 6. Deployment architecture. The quantised ONNX model is loaded by the ONNX Runtime and exposed through three integration points. Sample output shows line-level annotations.

The trained model is exported to ONNX format with 4-bit dynamic quantisation, reducing the file from approximately 130 MB (float32) to approximately 50 MB (int4). ONNX Runtime handles CPU inference at sub-150ms per diff on standard developer hardware. Three integration modes are planned:

VS Code Extension	Integrates with the VS Code Diagnostics API. Surfaces inline annotations as yellow (warning) or red (critical) markers on changed lines whenever a TypeScript file is saved.
CLI tool npx nextreview	Scans a directory and produces a structured JSON report or human-readable console output. Can be piped into other tools.
Pre-commit hook	Runs on staged diffs before each commit. Blocks the commit if any critical issues are detected; lists flagged lines with category and confidence score.

8 Evaluation Framework

Automatic metrics, human evaluation, ablation design

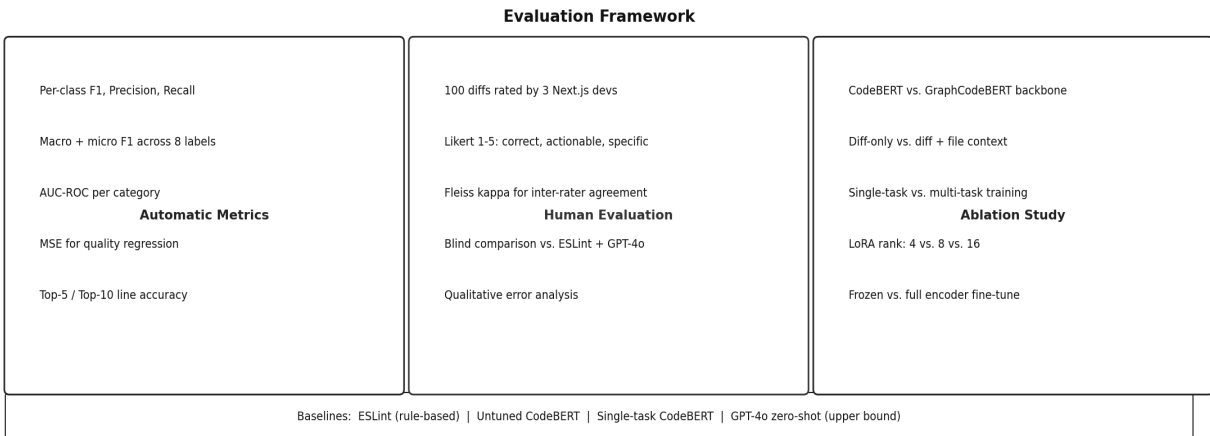


Figure 7. Evaluation framework with three pillars and four baselines.

Evaluation is structured across three complementary pillars. Automatic metrics provide objective comparison against baselines. Human evaluation establishes real-world validity with practising developers. Ablation studies attribute performance to specific architectural choices.

8.1 Automatic metrics

For the category head: per-class F1, precision, recall; macro and micro-averaged F1; AUC-ROC per category. For the quality regression head: MSE and Pearson correlation. For severity: accuracy and weighted F1. For the line-level head: Top-5 and Top-10 accuracy, following the evaluation methodology of LineVul (Fu & Tantithamthavorn, MSR 2022).

8.2 Human evaluation protocol

A stratified sample of 100 diffs is drawn from the held-out test set and presented to three independent Next.js developers with 3+ years of experience. Each rates the model output on a 5-point Likert scale across three dimensions: correctness, actionability, and localisation specificity. Fleiss' kappa measures inter-rater agreement. The same 100 diffs are also evaluated by ESLint and GPT-4o for direct comparison.

9 Project Timeline

Nine phases across approximately 19 weeks

Phase	Activity	Duration	Deliverable
1	Literature review and architecture finalisation	2 weeks	This document + literature review PDF
2	GitHub data collection pipeline	2 weeks	Raw PR dataset (~20K samples)
3	Cleaning, label extraction, train/val/test split	2 weeks	Annotated dataset (~15K samples)
4	Baseline: untuned CodeBERT on all four heads	1 week	Baseline metric report
5	LoRA fine-tuning and hyperparameter grid search	3 weeks	Best fine-tuned model checkpoint
6	Ablation study (backbone, input strategy, MTL)	2 weeks	Ablation results table
7	Human evaluation study	2 weeks	Human evaluation report
8	ONNX quantisation and deployment tooling	2 weeks	VS Code extension + CLI prototype
9	Paper writing and submission	3 weeks	Research paper draft

Table 4. Project timeline — nine phases, approximately 19 weeks total.